
通过深度强化学习玩 Atari

弗拉基米尔·姆尼·科雷·卡武克库奥格鲁 大卫·西尔弗 亚历克斯·格雷夫斯 扬尼斯·安东诺格鲁

达安·维尔斯特拉·马丁·里德米勒

深脑科技

{弗拉德、科雷、大卫、alex.graves、ioannis、daan、martin.riedmiller} @ deepmind.com

抽象的

我们提出了第一个深度学习模型，可以使用强化学习直接从高维感官输入成功学习控制策略。该模型是一个卷积神经网络，采用 Q 学习的变体进行训练，其输入是原始像素，其输出是估计未来奖励的值函数。我们将我们的方法应用于街机学习环境中的七款 Atari 2600 游戏，无需调整架构或学习算法。我们发现它在六款游戏中的表现优于之前的所有方法，并在其中三款游戏中超越了人类专家。

1 简介

学习直接从视觉和语音等高维感官输入控制智能体是强化学习（RL）长期面临的挑战之一。在这些领域运行的大多数成功的强化学习应用程序都依赖于手工制作的特征与线性价值函数或策略表示的结合。显然，此类系统的性能在很大程度上依赖于特征表示的质量。

深度学习的最新进展使得从原始感知数据中提取高级特征成为可能，从而在计算机视觉 [11, 22, 16] 和语音识别 [6, 7] 方面取得突破。这些方法利用了一系列神经网络架构，包括卷积网络、多层感知器、受限玻尔兹曼机和循环神经网络，并利用了监督学习和无监督学习。我们很自然地会问，类似的技术是否也有利于使用感知数据的强化学习。

然而，从深度学习的角度来看，强化学习提出了一些挑战。首先，迄今为止最成功的深度学习应用都需要大量手工标记的训练数据。另一方面，强化学习算法必须能够从通常稀疏、有噪声和延迟的标量奖励信号中学习。与监督学习中输入和目标之间的直接关联相比，行动和由此产生的奖励之间的延迟（可能长达数千个时间步长）似乎尤其令人畏惧。另一个问题是，大多数深度学习算法假设数据样本是独立的，而在强化学习中，通常会遇到高度相关状态的序列。此外，在强化学习中，随着算法学习新行为，数据分布会发生变化，这对于假设固定基础分布的深度学习方法来说可能会出现。

本文证明，卷积神经网络可以克服这些挑战，在复杂的 RL 环境中从原始视频数据中学习成功的控制策略。该网络使用 Q 学习 [26] 算法的变体进行训练，并使用随机梯度下降来更新权重。为了缓解相关数据和非平稳分布的问题，我们使用



图 1: 五款 Atari 2600 游戏的屏幕截图: (Left-to-right) Pong、Breakout、Space Invaders、Seaquest、Beam Rider

一种经验重放机制[13], 它随机采样先前的转换, 从而平滑许多过去行为的训练分布。

我们将我们的方法应用于在街机学习环境 (ALE) [3] 中实施的一系列 Atari 2600 游戏。Atari 2600 是一个具有挑战性的 RL 测试平台, 它为智能体提供高维视觉输入 (60Hz 的 210×160 RGB 视频) 以及一组多样化且有趣的任务, 这些任务被设计为对人类玩家来说是困难的。我们的目标是创建一个能够成功学习玩尽可能多的游戏的单一神经网络代理。该网络没有提供任何特定于游戏的信息或手工设计的视觉特征, 并且不了解模拟器的内部状态; 它只从视频输入、奖励和终端信号以及一组可能的动作中学习——就像人类玩家一样。此外, 网络架构和用于训练的所有超参数在整个游戏中保持不变。到目前为止, 该网络在我们尝试过的七款游戏中的六款中已经超越了之前所有的强化学习算法, 并在其中三款游戏中超越了人类专家玩家。图 1 提供了用于训练的五个游戏的示例屏幕截图。

2 背景

我们考虑代理与环境 $\mathcal{E}$ (在本例中为 Atari 模拟器) 交互的任务, 包括一系列动作、观察和奖励。在每个时间步, 代理从合法游戏动作集合 $\mathcal{A} = \{1, \dots, K\}$ 中选择一个动作 a_t 。该动作被传递到模拟器并修改其内部状态和游戏分数。一般来说, $\mathcal{E}$ 可能是随机的。代理无法观察到模拟器的内部状态; 相反, 它观察来自模拟器的图像 $x_t \in \mathbb{R}^d$, 它是表示当前屏幕的原始像素值的向量。此外, 它还收到代表游戏分数变化的奖励 r_t 。请注意, 一般来说, 游戏得分可能取决于之前的整个行动和观察序列; 有关某个操作的反馈可能只有在经过数千个时间步之后才能收到。

由于代理仅观察当前屏幕的图像, 因此任务被部分观察, 并且许多模拟器状态在感知上是混叠的, 即仅从当前屏幕 x_t 不可能完全理解当前情况。因此, 我们考虑动作和观察的序列 $s_t = x_1, a_1, x_2, \dots, a_{t-1}, x_t$, 并学习依赖于这些序列的游戏策略。假设模拟器中的所有序列都以有限数量的时间步终止。这种形式主义产生了一个大型但有限的马尔可夫决策过程 (MDP), 其中每个序列都是一个不同的状态。因此, 我们可以将标准的强化学习方法应用于 MDP, 只需使用完整的序列 s_t 作为时间 t 的状态表示即可。

代理的目标是通过以最大化未来奖励的方式选择操作来与模拟器进行交互。我们做出标准假设, 即未来奖励每个时间步按 γ 因子折扣, 并将时间 t 处的未来折扣 *return* 定义为 $R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$, 其中 T 是游戏终止的时间步。我们将最优动作值函数 $Q^*(s, a)$ 定义为在看到某些序列 s 并采取某些动作 a 、 $Q^*(s, a) = \max_{\pi} \mathbb{E}[R_t | s_t = s, a_t = a, \pi]$ 后, 遵循任何策略可实现的最大预期回报, 其中 π 是将序列映射到动作 (或动作上的分布) 的策略。

最佳动作-价值函数遵循一个重要的恒等式, 称为 *Bellman equation*。这是基于以下直觉: 如果已知所有可能动作 a' 的序列 s' 在下一个时间步的最优值 $Q^*(s', a')$, 则最优策略是选择动作 a'

最大化 $r + \gamma Q^*(s', a')$ 的期望值,

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{E}} \left[r + \gamma \max_{a'} Q^*(s', a') \middle| s, a \right] \quad (1)$$

许多强化学习算法背后的基本思想是通过使用贝尔曼方程作为迭代更新 $Q_{i+1}(s, a) = \mathbb{E} [r + \gamma \max_{a'} Q_i(s', a') | s, a]$ 来估计动作价值函数。这样的 *value iteration* 算法收敛到最佳动作价值函数, $Q_i \rightarrow Q^*$ 为 $i \rightarrow \infty$ [23]。在实践中, 这种基本方法是完全不切实际的, 因为动作价值函数是针对每个序列单独估计的, 没有任何概括。相反, 通常使用函数逼近器来估计动作价值函数 $Q(s, a; \theta) \approx Q^*(s, a)$ 。在强化学习社区中, 这通常是线性函数逼近器, 但有时会使用非线性函数逼近器, 例如神经网络。我们将权重为 θ 的神经网络函数逼近器称为 Q 网络。可以通过最小化每次迭代 i 时变化的损失函数 $L_i(\theta_i)$ 序列来训练 Q 网络,

$$L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot)} \left[(y_i - Q(s, a; \theta_i))^2 \right], \quad (2)$$

其中 $y_i = \mathbb{E}_{s' \sim \mathcal{E}} [r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) | s, a]$ 是迭代 i 的目标, $\rho(s, a)$ 是序列 s 和动作 a (我们称为 *behaviour distribution*) 的概率分布。优化损失函数 $L_i(\theta_i)$ 时, 上一次迭代 θ_{i-1} 的参数保持固定。请注意, 目标取决于网络权重; 这与监督学习使用的目标形成鲜明对比, 后者在学习开始之前就已确定。将损失函数与权重微分, 我们得到以下梯度:

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{s, a \sim \rho(\cdot); s' \sim \mathcal{E}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i) \right) \nabla_{\theta_i} Q(s, a; \theta_i) \right]. \quad (3)$$

通过随机梯度下降来优化损失函数通常在计算上是方便的, 而不是计算上述梯度中的全部期望。如果在每个时间步之后更新权重, 并且分别用来自行为分布 ρ 和模拟器 $\mathcal{E}$ 的单个样本替换期望, 那么我们就得到了熟悉的 *Q-learning* 算法 [26]。

请注意, 该算法是 *model-free*: 它直接使用来自模拟器 $\mathcal{E}$ 的样本解决强化学习任务, 而不显式构建 $\mathcal{E}$ 的估计。它也是 *off-policy*: 它学习贪婪策略 $a = \max_a Q(s, a; \theta)$, 同时遵循确保充分探索状态空间的行为分布。在实践中, 行为分布通常是通过 ϵ -贪婪策略来选择的, 该策略遵循概率为 $1 - \epsilon$ 的贪婪策略, 并以概率 ϵ 选择随机动作。

3 相关工作

也许强化学习最著名的成功故事是 *TD-gammon*, 这是一个西洋双陆棋游戏程序, 完全通过强化学习和自我对弈来学习, 并达到了超人的游戏水平 [24]。TD-gammon 使用了一种类似于 Q-learning 的无模型强化学习算法, 并使用具有一个隐藏层¹的多层感知器来近似价值函数。

然而, 早期尝试跟进 TD-gammon, 包括将相同的方法应用于国际象棋、围棋和跳棋, 但不太成功。这导致人们普遍认为 TD-gammon 方法是一种特殊情况, 仅适用于双陆棋, 也许是因为掷骰子的随机性有助于探索状态空间, 并使价值函数特别平滑 [19]。

此外, 研究表明, 将无模型强化学习算法 (例如 Q 学习) 与非线性函数逼近器 [25] 或实际上与离策略学习 [1] 相结合可能会导致 Q 网络发散。随后, 强化学习的大部分工作都集中在具有更好收敛保证的线性函数逼近器上 [25]。

¹In fact TD-Gammon approximated the state value function $V(s)$ rather than the action-value function $Q(s, a)$, and learnt *on-policy* directly from the self-play games

最近，人们对将深度学习与强化学习相结合重新产生了兴趣。深度神经网络已被用来估计环境 $\mathcal{E}$ ；受限玻尔兹曼机已被用来估计价值函数[21]；或政策[9]。此外，Q 学习的发散问题已通过 *gradient temporal-difference* 方法得到部分解决。这些方法被证明在使用非线性函数逼近器评估固定策略时是收敛的[14]；或者使用 Q 学习的受限变体通过线性函数逼近学习控制策略时[15]。然而，这些方法尚未扩展到非线性控制。

也许与我们自己的方法最相似的先前工作是神经拟合 Q 学习 (NFQ) [20]。NFQ 优化了等式 2 中的损失函数序列，使用 RPROP 算法来更新 Q 网络的参数。然而，它使用批量更新，每次迭代的计算成本与数据集的大小成正比，而我们考虑随机梯度更新，每次迭代的恒定成本较低，并且可以扩展到大型数据集。NFQ 还已成功应用于使用纯视觉输入的简单现实世界控制任务，首先使用深度自动编码器学习任务的低维表示，然后将 NFQ 应用于该表示[12]。相比之下，我们的方法直接从视觉输入中应用端到端的强化学习；因此，它可以学习与区分动作值直接相关的特征。Q-learning 之前也曾与经验回放和简单的神经网络 [13] 相结合，但同样是从低维状态而不是原始视觉输入开始。

[3] 介绍了使用 Atari 2600 模拟器作为强化学习平台，该平台应用了具有线性函数近似和通用视觉特征的标准强化学习算法。随后，通过使用更多的特征，并使用河哈希将特征随机投影到较低维的空间中，结果得到了改进[2]。HyperNEAT 进化架构 [8] 也已应用于 Atari 平台，用于进化（针对每个不同的游戏单独）代表该游戏策略的神经网络。当使用模拟器的重置工具针对确定性序列进行反复训练时，这些策略能够利用多个 Atari 游戏中的设计缺陷。

4 深度强化学习

计算机视觉和语音识别领域的最新突破依赖于在非常大的训练集上有效地训练深度神经网络。最成功的方法是直接从原始输入进行训练，使用基于随机梯度下降的轻量级更新。通过将足够的数据输入深度神经网络，通常可以学习比手工特征更好的表示[11]。这些成功激发了我们强化学习的方法。我们的目标是将强化学习算法连接到直接在 RGB 图像上运行的深度神经网络，并通过使用随机梯度更新有效地处理训练数据。

Tesauro 的 TD-Gammon 架构为这种方法提供了一个起点。该架构直接根据策略经验样本 $s_t, a_t, r_t, s_{t+1}, a_{t+1}$ 更新估计价值函数的网络参数，这些经验样本来自算法与环境的交互（或者通过自我对弈，在双陆棋的情况下）。由于这种方法在 20 年前就能够超越人类最好的双陆棋玩家，因此很自然地想知道二十年的硬件改进，再加上现代深度神经网络架构和可扩展的 RL 算法是否会产生显着的进步。

与 TD-Gammon 和类似的在线方法相比，我们利用一种称为 *experience replay* [13] 的技术，其中我们将代理在每个时间步的经验 $e_t = (s_t, a_t, r_t, s_{t+1})$ 存储在数据集 $\mathcal{D} = e_1, \dots, e_N$ 中，并将许多情节汇集到 *replay memory* 中。在算法的内循环期间，我们将 Q 学习更新或小批量更新应用于从存储样本池中随机抽取的经验样本 $e \sim \mathcal{D}$ 。执行经验重放后，代理根据 ϵ 贪婪策略选择并执行操作。由于使用任意长度的历史作为神经网络的输入可能很困难，因此我们的 Q 函数改为使用函数 ϕ 生成的历史的固定长度表示。完整的算法，我们称之为 *deep Q-learning*，如算法 1 所示。

这种方法比标准在线 Q-learning [23] 有几个优点。首先，每一步的经验都可能用于许多权重更新，从而提高数据效率。

Algorithm 1 Deep Q-learning with Experience Replay

```
Initialize replay memory  $\mathcal{D}$  to capacity  $N$ 
Initialize action-value function  $Q$  with random weights
for episode = 1,  $M$  do
  Initialise sequence  $s_1 = \{x_1\}$  and preprocessed sequenced  $\phi_1 = \phi(s_1)$ 
  for  $t = 1, T$  do
    With probability  $\epsilon$  select a random action  $a_t$ 
    otherwise select  $a_t = \max_a Q^*(\phi(s_t), a; \theta)$ 
    Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
    Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$ 
    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$ 
    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  according to equation 3
  end for
end for
```

其次，由于样本之间的相关性很强，直接从连续样本中学习是低效的。随机化样本会破坏这些相关性，从而减少更新的方差。第三，在同策略学习时，当前参数决定参数训练的下一个数据样本。例如，如果最大化动作是向左移动，那么训练样本将来自左侧的样本为主；如果最大化动作随后切换到右侧，那么训练分布也会切换。很容易看出可能会出现不需要的反馈循环，并且参数可能会陷入不良的局部最小值，甚至灾难性地发散[25]。通过使用经验重播，行为分布会在其之前的许多状态上进行平均，从而平滑学习并避免参数的振荡或发散。请注意，当通过经验回放进行学习时，需要学习 off-policy（因为我们当前的参数与用于生成样本的参数不同），这激发了 Q-learning 的选择。

实际上，我们的算法仅将最后的 N 经验元组存储在重放内存中，并在执行更新时从 $\mathcal{D}$ 中均匀随机采样。这种方法在某些方面受到限制，因为内存缓冲区不区分重要的转换，并且由于有限的内存大小 N 总是用最近的转换覆盖。类似地，均匀采样对重放存储器中的所有转换给予同等的重要性。更复杂的采样策略可能会强调我们可以从中学到最多的转变，类似于优先扫描[17]。

4.1 预处理和模型架构

直接处理原始 Atari 帧（即具有 128 个调色板的 210×160 像素图像）的计算要求可能很高，因此我们应用了旨在降低输入维度的基本预处理步骤。通过首先将 RGB 表示转换为灰度并将其下采样为 110×84 图像来对原始帧进行预处理。最终的输入表示是通过裁剪图像的 84×84 区域来获得的，该区域大致捕获了游戏区域。仅需要最后的裁剪阶段，因为我们使用[11]中的 2D 卷积的 GPU 实现，它需要平方输入。对于本文的实验，算法 1 中的函数 ϕ 将此预处理应用于历史记录的最后 4 帧，并将它们堆叠起来以生成 Q 函数的输入。

使用神经网络参数化 Q 有多种可能的方法。由于 Q 将历史-动作对映射到其 Q 值的标量估计，因此历史和动作已被一些先前的方法用作神经网络的输入 [20, 12]。这种类型的架构的主要缺点是需要单独的前向传递来计算每个动作的 Q 值，从而导致成本随动作数量线性扩展。相反，我们使用一种架构，其中每个可能的动作都有一个单独的输出单元，并且只有状态表示是神经网络的输入。输出对应于输入状态的单个动作的预测 Q 值。这种类型的架构的主要优点是能够计算给定状态下所有可能动作的 Q 值，而只需通过网络进行一次前向传递。

我们现在描述所有七款 Atari 游戏所使用的确切架构。神经网络的输入是由 ϕ 生成的 $84 \times 84 \times 4$ 图像。第一个隐藏层将步幅为 4 的 $16 \times 8 \times 8$ 滤波器与输入图像进行卷积，并应用整流非线性 [10, 18]。第二个隐藏层以步长 2 与 $32 \times 4 \times 4$ 个滤波器进行卷积，后面同样是非线性整流器。最终隐藏层是全连接的，由 256 个整流单元组成。输出层是一个全连接的线性层，每个有效动作都有一个输出。在我们考虑的游戏里，有效动作的数量在 4 到 18 之间变化。我们将使用我们的方法训练的卷积网络称为深度 Q 网络 (DQN)。

5 实验

到目前为止，我们已经对七款流行的 ATARI 游戏进行了实验——Beam Rider、Breakout、Enduro、Pong、Q*bert、Seaquest、Space Invaders。我们在所有七款游戏中使用相同的网络架构、学习算法和超参数设置，这表明我们的方法足够强大，可以在不合并游戏特定信息的情况下适用于各种游戏。当我们在真实且未经修改的游戏中评估我们的代理时，我们仅在训练期间对游戏的奖励结构进行了一项更改。由于不同游戏的分数大小差异很大，我们将所有正奖励固定为 1，所有负奖励固定为 -1，保留 0 奖励不变。以这种方式削减奖励可以限制误差导数的规模，并且可以更轻松地在多个游戏中使用相同的学习率。同时，它可能会影响我们代理的表现，因为它无法区分不同大小的奖励。

在这些实验中，我们使用大小为 32 的小批量的 RMSProp 算法。训练期间的行为策略是 ϵ 贪婪，其中 ϵ 在前一百万帧中从 1 线性退火到 0.1，此后固定为 0.1。我们总共训练了 1000 万帧，并使用了最近 100 万帧的重放内存。

遵循之前玩 Atari 游戏的方法，我们还使用了简单的跳帧技术 [3]。更准确地说，代理在每个 k^{th} 帧而不是每个帧上查看并选择动作，并且在跳过的帧上重复其最后一个动作。由于向前运行模拟器一步所需的计算量比让代理选择动作要少得多，因此该技术允许代理玩大约 k 倍的游戏，而不会显著增加运行时间。我们对除 Space Invaders 之外的所有游戏都使用 $k = 4$ ，我们注意到使用 $k = 4$ 会使激光不可见，因为它们闪烁的周期不同。我们使用 $k = 3$ 使激光可见，这一更改是所有游戏之间超参数值的唯一差异。

5.1 训练和稳定性

在监督学习中，人们可以通过在训练和验证集上评估模型来轻松跟踪模型在训练期间的性能。然而，在强化学习中，准确评估智能体在训练过程中的进度可能具有挑战性。正如 [3] 所建议的，我们的评估指标是智能体在一个回合或一场比赛中收集的总奖励，在多场比赛中平均，因此我们在训练期间定期计算它。平均总奖励指标往往非常嘈杂，因为策略权重的微小变化可能会导致策略访问的状态分布发生巨大变化。图 2 中最左边的两个图显示了 Seaquest 和 Breakout 游戏训练期间平均总奖励如何变化。两个平均奖励图确实都相当嘈杂，给人一种学习算法没有取得稳定进展的印象。另一个更稳定的指标是策略的估计动作价值函数 Q ，它提供了代理在任何给定状态下通过遵循其策略可以获得多少折扣奖励的估计。我们在训练开始之前通过运行随机策略来收集一组固定的状态，并跟踪这些状态的最大预测值²的平均值。图 2 中最右边的两个图显示，平均预测 Q 的增加比智能体获得的平均总奖励平滑得多，并且在其他五场游戏上绘制相同的指标会产生类似的平滑曲线。除了在训练期间看到预测的 Q 得到相对平稳的改进之外，我们在任何实验中都没有遇到任何发散问题。这表明，尽管缺乏任何理论收敛保证，我们的方法能够以稳定的方式使用强化学习信号和随机梯度下降来训练大型神经网络。

²The maximum for each state is taken over the possible actions.

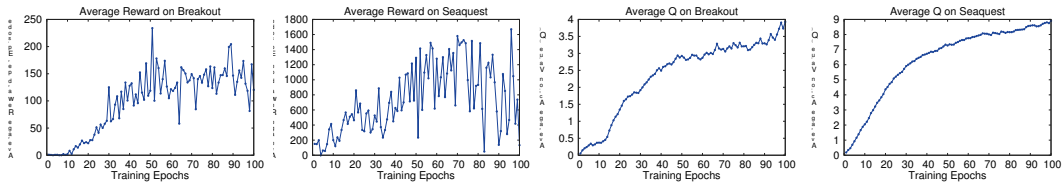


图 2: 左侧的两个图分别显示了训练期间 Breakout 和 Seaquest 中每集的平均奖励。统计数据是通过运行 $\epsilon = 0.05$ 的 ϵ 贪婪策略 10000 步来计算的。右侧的两个图分别显示了 Breakout 和 Seaquest 上一组状态的平均最大预测动作值。一个 epoch 对应于 50000 个小批量权重更新或大约 30 分钟的训练时间。



图 3: 最左边的图显示了游戏 Seaquest 的 30 帧片段的预测值函数。这三个屏幕截图分别对应于 A、B、C 标记的帧。

5.2 价值函数可视化

图 3 显示了游戏 Seaquest 中学习价值函数的可视化。从图中可以看出，当敌人出现在屏幕左侧 (A 点) 后，预测值会发生跳跃。然后，代理向敌人发射鱼雷，当鱼雷即将击中敌人时，预测值达到峰值 (B 点)。最后，在敌人消失后，该值大致下降到原来的值 (C 点)。图 3 表明我们的方法能够了解价值函数如何随着相当复杂的事件序列而演变。

5.3 主要评价

我们将我们的结果与 RL 文献 [3, 4] 中表现最佳的方法进行比较。标记为 Sarsa 的方法使用 Sarsa 算法来学习针对 Atari 任务手工设计的几个不同特征集的线性策略，我们报告了最佳性能特征集的得分 [3]。意外事件使用与 Sarsa 相同的基本方法，但通过学习代理控制下的屏幕部分的表示来增强特征集 [4]。请注意，这两种方法都通过使用背景扣除并将 128 种颜色中的每一种都视为单独的通道，结合了有关视觉问题的重要先验知识。由于许多 Atari 游戏对每种类型的对象使用一种不同的颜色，因此将每种颜色视为单独的通道可能类似于生成对每种对象类型的存在进行编码的单独的二进制映射。相比之下，我们的代理仅接收原始 RGB 屏幕截图作为输入，并且必须 *learn* 来自行检测对象。

除了学习到的代理之外，我们还报告了专家人类游戏玩家的分数以及随机统一选择动作的策略。人类表现是玩每场游戏大约两个小时后获得的中值奖励。请注意，我们报告的人类得分远高于 Bellemare 等人的得分。[3]。对于学习方法，我们遵循 Bellemare 等人使用的评估策略。[3, 5] 并报告通过运行具有 $\epsilon = 0.05$ 的 ϵ 贪婪策略固定步数获得的平均分数。表 1 的前五行显示了所有比赛的每场平均得分。我们的方法 (标记为 DQN) 在所有七款游戏中都远远优于其他学习方法，尽管几乎没有包含有关输入的先验知识。

我们还在表 1 的最后三行中与 [8] 中的进化策略搜索方法进行了比较。我们报告了该方法的两组结果。HNeat Best 分数反映了使用手工设计的对象检测器算法获得的结果，该算法输出位置和

	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	-20.4	157	110	179
Sarsa [3]	996	5.2	129	-19	614	665	271
Contingency [4]	1743	6	159	-17	960	723	268
DQN	4092	168	470	20	1952	1705	581
Human	7456	31	368	-3	18900	28010	3690
HNeat Best [8]	3616	52	106	19	1800	920	1720
HNeat Pixel [8]	1332	4	91	-16	1325	800	1145
DQN Best	5184	225	661	21	4500	1740	1075

表 1: 上表比较了不同学习方法的平均总奖励, 通过运行 ϵ 贪婪策略 ($\epsilon = 0.05$) 固定步数。下表报告了 HNeat 和 DQN 的单个最佳表现片段的结果。HNeat 生成始终获得相同分数的确定性策略, 而 DQN 使用 ϵ 贪婪策略, 其中 $\epsilon = 0.05$ 。

Atari 屏幕上的对象类型。HNeat Pixel 分数是通过使用 Atari 模拟器的特殊 8 颜色通道表示获得的, 该表示表示每个通道的对象标签图。该方法在很大程度上依赖于找到代表成功利用的确定性状态序列。以这种方式学习的策略不太可能推广到随机扰动; 因此, 该算法仅针对得分最高的单集进行评估。相反, 我们的算法是在 ϵ -贪婪控制序列上进行评估的, 因此必须概括各种可能的情况。尽管如此, 我们表明, 在除 Space Invaders 之外的所有游戏中, 不仅我们的最大评估结果 (第 8 行), 而且我们的平均结果 (第 4 行) 都取得了更好的性能。

最后, 我们证明了我们的方法在 Breakout、Enduro 和 Pong 上比人类专家玩家取得了更好的性能, 并且在 Beam Rider 上达到了接近人类的性能。Q*bert、Seaquest、Space Invaders 等游戏的表现与人类水平相去甚远, 但它们更具挑战性, 因为它们要求网络找到一种可在较长时间尺度上扩展的策略。

6 结论

本文介绍了一种新的强化学习深度学习模型, 并展示了其仅使用原始像素作为输入来掌握 Atari 2600 电脑游戏的困难控制策略的能力。我们还提出了一种在线 Q 学习的变体, 它将随机小批量更新与经验回放记忆相结合, 以简化强化学习深度网络的训练。我们的方法在测试的七款游戏中的六款中给出了最先进的结果, 无需调整架构或超参数。

参考

- [1] 莱蒙·贝尔德。残差算法: 使用函数逼近的强化学习。在 *Proceedings of the 12th International Conference on Machine Learning (ICML 1995)* 中, 第 30-37 页。摩根·考夫曼, 1995。[2] 马克·贝尔马尔、乔尔·维内斯和迈克尔·鲍林。基于草图的线性值函数近似。 *Advances in Neural Information Processing Systems* 25, 第 2222-2230 页, 2012 年。[3] Marc G Bellemare、Yavar Naddaf、Joel Veness 和 Michael Bowling。街机学习环境: 通用智能体的评估平台。 *Journal of Artificial Intelligence Research*, 47: 253-279, 2013。[4] 马克·G·贝勒马尔、乔尔·维内斯和迈克尔·鲍林。使用 Atari 2600 游戏调查应急意识。 AAAI, 2012 年。[5] Marc G. Bellemare、Joel Veness 和 Michael Bowling。递归因子环境的贝叶斯学习。 *Proceedings of the Thirtieth International Conference on Machine Learning (ICML 2013)*, 第 1211-1219 页, 2013 年。

[6] 乔治·E·达尔 (George E. Dahl)、于冬、邓力、亚历克斯·阿塞罗 (Alex Acero)。用于大词汇量语音识别的上下文相关预训练深度神经网络。 *Audio, Speech, and Language Processing, IEEE Transactions on*, 20(1):30–42, 2012 年 1 月。[7] Alex Graves、Abdel-rahman Mohamed 和 Geoffrey E. Hinton。使用深度循环神经网络进行语音识别。 *Proc. ICASSP*, 2013 年。[8] Matthew Hausknecht、Risto Miikkulainen 和 Peter Stone。一般 Atari 游戏的神经进化方法。2013。[9] 尼古拉斯·赫斯、大卫·西尔弗和 Yee Whye Teh。具有基于能量的策略的演员批评家强化学习。见 *European Workshop on Reinforcement Learning*, 第 43 页, 2012 年。[10] Kevin Jarrett、Koray Kavukcuoglu、Marc Aurelio Ranzato 和 Yann LeCun。物体识别的最佳多级架构是什么? 在 *Proc. International Conference on Computer Vision and Pattern Recognition (CVPR 2009)*, 第 2146–2153 页。IEEE, 2009。[11] Alex Krizhevsky、Ilya Sutskever 和 Geoff Hinton。使用深度卷积神经网络进行 Imagenet 分类。 *Advances in Neural Information Processing Systems 25*, 第 1106–1114 页, 2012 年。[12] Sasha Lange 和 Martin Riedmiller。强化学习中的深度自动编码器神经网络。在 *Neural Networks (IJCNN), The 2010 International Joint Conference on* 中, 第 1–8 页。IEEE, 2010。[13] 林龙吉。使用神经网络的机器人强化学习。技术报告, DTIC 文件, 1993 年。[14] Hamid Maei、Csaba Szepesvari、Shalabh Bhatnagar、Doina Precup、David Silver 和 Rich Sutton。具有任意平滑函数逼近的收敛时间差分学习。见 *Advances in Neural Information Processing Systems 22*, 第 1204–1212 页, 2009 年。[15] Hamid Maei、Csaba Szepesvári、Shalabh Bhatnagar 和 Richard S. Sutton。通过函数逼近实现离策略学习控制。见 *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*, 第 719–726 页, 2010 年。[16] Volodymyr Mnih。 *Machine Learning for Aerial Image Labeling*。博士论文, 多伦多大学, 2013 年。[17] Andrew Moore 和 Chris Atkeson。优先清扫: 数据较少、实时性较差的强化学习。 *Machine Learning*, 13: 103–130, 1993。[18] 维诺德·奈尔和杰弗里·E·辛顿。整流线性单元改进了受限玻尔兹曼机。 *Proceedings of the 27th International Conference on Machine Learning (ICML 2010)*, 第 807–814 页, 2010 年。[19] Jordan B. Pollack 和 Alan D. Blair。为什么 td-gammon 有效。见 *Advances in Neural Information Processing Systems 9*, 第 10–16 页, 1996 年。[20] Martin Riedmiller。神经拟合 q 迭代——数据高效神经强化学习方法的首次体验。在 *Machine Learning: ECML 2005* 中, 第 317–328 页。施普林格, 2005 年。[21] Brian Sallans 和 Geoffrey E. Hinton。具有分解状态和动作的强化学习。 *Journal of Machine Learning Research*, 5:1063–1088, 2004。[22] Pierre Sermanet、Koray Kavukcuoglu、Soumith Chintala 和 Yann LeCun。具有无监督多阶段特征学习的行人检测。在 *Proc. International Conference on Computer Vision and Pattern Recognition (CVPR 2013)* 中。IEEE, 2013。[23] 理查德·萨顿和安德鲁·巴托。 *Reinforcement Learning: An Introduction*。麻省理工学院出版社, 1998。[24] 杰拉尔德·特索罗。时间差异学习和 td-gammon。 *Communications of the ACM*, 38(3):58–68, 1995。[25] 约翰·N·齐齐克利斯 (John N Tsitsiklis) 和本杰明·范·罗伊 (Benjamin Van Roy)。函数逼近的时差学习分析。 *Automatic Control, IEEE Transactions on*, 42(5): 674–690, 1997。[26] 克里斯托弗·J·沃特金斯和彼得·达扬。Q-学习。 *Machine learning*, 8(3–4): 279–292, 1992。